

★ WEEK 1 · PROBLEM SET

Finance Applications: Three Hands-On Problems

Fat Tails · Overfitting · Credit Default

Each problem: Statement → Principles & Lab Guide → Hand Calculation → Claude Code Prompt

Small numbers you can compute with a pencil — then a metaprompt that lets Claude Code scale the same idea to real market data. Hand first, machine second.

Every Problem Follows the Same Four Steps

Work top to bottom — do not skip the hand calculation, it is where the understanding lives

1 · Statement

the question

A concrete finance scenario with a tiny dataset you can see at a glance.

one slide

2 · Principles

the tools

The formulas involved and a short lab guide: what to compute, in what order.

one slide

3 · Hand Calc

pencil first

Work every number by hand in a table.
Small n , clean arithmetic, no calculator fear.

1–2 slides

4 · Claude Code

scale it up

A metaprompt: paste it into Claude Code and it builds the full lab on real data.

one slide

Hand first, machine second — you check the machine, never the other way around.

Where the Data Comes From

Public sources first — the prompts fetch free data; CSV files are offline fallbacks only

Problem	Primary source (public · no signup)	Provided fallback file
P1 · Fat tails	yfinance — real AAPL / 005930.KS daily prices	fml_w1_p1_returns_offline.csv
P2 · Overfitting	Simulated inside the script (numpy, fixed seed)	— none needed
P3 · Credit default	Simulation + real OpenML 'credit-g' via scikit-learn	fml_w1_p3_credit_sim.csv
Hand calculations	Printed on the slides (5 returns · 8 borrowers)	fml_w1_p3_borrowers.csv

Course rule: if data is free online (yfinance, FRED, OpenML), the prompt fetches it — files are for offline classrooms.

P1

PROBLEM 1 • FAT TAILS

Are Daily Returns a Nice Bell Curve?

Skewness, kurtosis, and one crash day that ruins the picture

One Crash Day in Five

A stock posts these five daily returns. Is this what a bell curve would produce?

Compute σ , skewness and kurtosis of these five returns by hand. Then decide: are they normal — and which day decides the answer?

Day	Mon	Tue	Wed	Thu	Fri
Return r (%)	-4	+1	+1	+1	+1

Hint: the mean is deliberately a round number — check it first.

Your deliverables

- Mean, variance, and σ of the five returns.
- Skewness S and kurtosis K (and $K - 3$).
- A one-line verdict: fat tails — yes or no?
- Then: verify on 2 years of real data via Claude Code.

The Tools: Third and Fourth Moments

Skewness asks 'which tail is longer?' — kurtosis asks 'how heavy are the tails?'

$$S = \frac{\frac{1}{n} \sum_{i=1}^n (r_i - \bar{r})^3}{\sigma^3}$$

$$K = \frac{\frac{1}{n} \sum_{i=1}^n (r_i - \bar{r})^4}{\sigma^4}, \quad K_{\text{excess}} = K - 3$$

Reading the two numbers

- $S < 0$: the long tail is on the LEFT — crashes.
- $K = 3$ for a perfect normal; $K - 3$ is 'excess'.
- $K - 3 > 0$: extreme days come more often than normal allows.

Lab guide — four steps

- Step 1: mean $\bar{r}$, then deviations $(r - \bar{r})$.
- Step 2: square, cube, fourth-power each; sum columns.
- Step 3: σ^2 from squares; then S and K .
- Step 4: compare S to 0 and K to 3.

The Moment Table — Every Number by Hand

Mean = $(-4+1+1+1+1)/5 = 0$, so deviations equal the returns themselves

Day	r	$(r - \bar{r})$	$(r - \bar{r})^2$	$(r - \bar{r})^3$	$(r - \bar{r})^4$
Mon	-4	-4	16	-64	256
Tue	+1	+1	1	+1	1
Wed	+1	+1	1	+1	1
Thu	+1	+1	1	+1	1
Fri	+1	+1	1	+1	1
Σ (sum)	0	0	20	-60	260

$$\sigma^2 = \frac{1}{n} \sum (r_i - \bar{r})^2 = \frac{20}{5} = 4 \Rightarrow \sigma = 2\%$$

One crash day (Mon) contributes 16 of 20 squares, -64 of -60 cubes, 256 of 260 fourth powers.

Verdict: Left-Skewed, Slightly Fat-Tailed

Plug the sums into the formulas — the crash day dominates every moment

$$S = \frac{-60/5}{2^3} = \frac{-12}{8} = -1.5 \quad K = \frac{260/5}{2^4} = \frac{52}{16} = 3.25$$

$$JB = \frac{5}{6} \left((-1.5)^2 + \frac{0.25^2}{4} \right) \approx 1.89 < 5.99 = \chi_{2,5\%}^2$$

What it means

- $S = -1.5$: strongly left-skewed — risk lives in the crash tail.
- $K = 3.25$ (excess +0.25): mildly fatter than normal.
- One day out of five moved every moment — that IS financial data.

Caveat — sample size

- $JB = 1.89 < 5.99$: with $n = 5$ we cannot formally reject normality.
- Moments need data — real daily series show excess kurtosis of 5–10.
- That is what the Claude Code step will show.

Scale It: From Five Returns to Two Years

Paste the metaprompt — Claude Code builds the full fat-tails lab on real data

[INPUT] Prompt · Claude Code

```
Build a single Colab-ready Python script that:
1) downloads 2 years of daily prices for AAPL and
   005930.KS with yfinance and computes daily returns;
2) for each ticker computes skewness, excess kurtosis
   and the Jarque-Bera statistic (scipy.stats);
3) plots, side by side, a return histogram with a
   fitted normal curve and a Q-Q plot;
4) prints a small table: ticker | skew | ex.kurt | JB,
   plus a one-line verdict 'fat tails: YES/NO' per ticker.
If the download fails (offline), load the provided
fml_w1_p1_returns_offline.csv instead (same analysis).
Comment every block in one plain-English line.
Use only numpy, scipy, matplotlib, yfinance, pandas.
```

[OUTPUT] What to expect

```
skew      : sign varies
ex. kurt   : clearly > 0 (5+)
JB        : hundreds to 1000s
verdict    : fat tails: YES
```

Kurtosis matches your n=5
result. Skew need not — one
crash in five is not 2 years.

P2

PROBLEM 2 · OVERFITTING

Catch the Memorizer with Numbers

Train MSE vs. test MSE — Student A exposed in one small table

Two Models, Four Training Days, One Test Day

Model A is a straight line; Model B memorizes every point. Which do you trust on day 5?

Using days 1–4 as the training set and day 5 as the test set, compute the train and test MSE of both models — by hand.

x (day)	1	2	3	4	5
y (price move)	2	3	5	6	7
Role	train	train	train	train	TEST

$$\hat{y}_A(x) = x + 1$$

$$\hat{y}_B(x) = \text{cubic through all 4 points}$$

Your deliverables

- Model A predictions and train MSE.
 - Model B: show train MSE = 0, then extrapolate it to day 5.
 - Test MSE for both · name the winner.
- o Bias² + Variance + Noise.

The Tools: MSE and Its Decomposition

One number to measure mistakes — and the three sources it hides

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$\mathbb{E}[(y - \hat{y})^2] = \underbrace{\text{Bias}^2}_{\text{too simple}} + \underbrace{\text{Variance}}_{\text{too jumpy}} + \underbrace{\sigma_\varepsilon^2}_{\text{pure noise}}$$

Lab guide — the protocol

- Fit using training points ONLY — the test point stays locked away.
- Measure MSE separately on train and on test.
- The train-test GAP, not the train score, is the diagnosis.

For Model B (the memorizer)

- A cubic through 4 points fits them exactly $\rightarrow$ train MSE = 0.
- Extrapolate with a difference table: extend Δ , Δ^2 , Δ^3 one step.
- No calculus needed — just additions.

Two Small Tables Settle It

Model A: $\hat{y} = x + 1$. Model B: the cubic through all four training points

x	y	$\hat{y}_A = x+1$	err^2_A	$\hat{y}_B$ (cubic)	err^2_B
1	2	2	0	2	0
2	3	3	0	3	0
3	5	4	1	5	0
4	6	5	1	6	0
5 (TEST)	7	6	1	4	9

Model B on day 5

- $y: 2, 3, 5, 6 \rightarrow \Delta: 1, 2, 1 \rightarrow \Delta^2: 1, -1 \rightarrow \Delta^3: -2$.
- Extend: Δ^2 next = -3, Δ next = -2.
- So $\hat{y}_B(5) = 6 - 2 = 4$, but $y = 7 \rightarrow \text{err}^2 = 9$.

Train MSE: $A = (0+0+1+1)/4 = 0.50$ · $B = 0.00$. Test MSE: $A = 1.00$ · $B = 9.00$.

Perfect in Training, Nine Times Worse in Testing

The memorizer wins the homework and loses the exam

0.50

Model A · train MSE —
honest, slightly
imperfect

0.00

Model B · train MSE —
suspiciously perfect

1.00

Model A · test MSE —
holds up on the future

9.00

Model B · test MSE —
the memorizer
exposed

Read it with Week 1 language

- B: low bias, huge variance — the wiggly cubic is Student A.
- A accepts a little bias and buys stability — the better trade.

The finance translation

- Backtest = training set; live trading = test set.
- The most beautiful backtests are often the most overfit.

Scale It: The Full U-Shaped Test Curve

Paste the metaprompt — Claude Code sweeps every polynomial degree for you

[INPUT] Prompt · Claude Code

```
Build a single Colab-ready Python script that:  
1) simulates  $y = \sin(2\pi x) + \text{Gaussian noise (sd 0.2)}$   
   on 30 points, and splits them randomly 70/30 into  
   train and test sets (fixed seed for reproducibility);  
2) fits polynomial models of degree 1 through 15 on the  
   training set only (numpy.polyfit);  
3) computes train MSE and test MSE for every degree and  
   plots both curves on one chart, marking the degree  
   with the lowest TEST error;  
4) prints a table degree | train MSE | test MSE and a  
   one-line verdict naming the sweet-spot degree.  
Comment every block in one plain-English line.
```

[OUTPUT] What to expect

```
train MSE: falls forever  
test  MSE: U-shape,  
           minimum near degree 3-5  
degree 15: test MSE explodes
```

```
Same story as your table:  
train score flatters,  
test score tells the truth.
```

P3

PROBLEM 3 · CREDIT DEFAULT

Score Eight Borrowers by Hand

A linear risk score, a confusion matrix, and why accuracy can lie

A Bank's One-Line Scoring Rule

Score $z = 2 \cdot \text{Debt} - \text{Income}$ (both in \$100k). Predict default when $z > 0$. How good is this rule?

Applicant	Income	Debt	Actually defaulted?
A	1.0	0.8	Yes
B	2.0	0.5	No
C	0.8	0.9	Yes
D	1.5	1.0	No
E	1.2	0.4	No
F	0.9	0.5	Yes
G	1.1	0.5	Yes
H	2.5	0.9	No

Your deliverables

- z score and prediction for all eight.
- The 2×2 confusion matrix (TP, FP, FN, TN).
- Accuracy, precision, recall — by hand.
- One sentence: which mistake costs the bank more, and which metric watches it?

This is exactly the 'risk = $2 \times \text{debt} - 1 \times \text{income}$ ' rule from the lecture, taken seriously.

The Tools: A Linear Score and Three Metrics

Classification is scoring plus a threshold — evaluation is counting four kinds of outcome

$$z_i = 2 \cdot \text{Debt}_i - 1 \cdot \text{Income}_i \quad \hat{y}_i = \text{Default} \iff z_i > 0$$

$$\text{Accuracy} = \frac{TP+TN}{n} \quad \text{Precision} = \frac{TP}{TP+FP} \quad \text{Recall} = \frac{TP}{TP+FN}$$

The four outcomes

- TP: predicted default, did default. TN: predicted safe, was safe.
- FP: false alarm — a good customer refused.
FN: a defaulter approved.

Lab guide — three steps

- Compute z per applicant; predict Default iff $z > 0$.
- Tally TP / FP / FN / TN against the actual column, then plug into the three formulas.

Scoring All Eight Applicants

One row per applicant: score, prediction, actual, outcome

Applicant	$z = 2 \cdot \text{Debt} - \text{Income}$	Predict ($z > 0$?)	Actual	Outcome
A	$1.6 - 1.0 = +0.6$	Default	Yes	TP
B	$1.0 - 2.0 = -1.0$	Safe	No	TN
C	$1.8 - 0.8 = +1.0$	Default	Yes	TP
D	$2.0 - 1.5 = +0.5$	Default	No	FP (false alarm)
E	$0.8 - 1.2 = -0.4$	Safe	No	TN
F	$1.0 - 0.9 = +0.1$	Default	Yes	TP
G	$1.0 - 1.1 = -0.1$	Safe	Yes	FN (missed default)
H	$1.8 - 2.5 = -0.7$	Safe	No	TN

F (+0.1) and G (-0.1) sit right at the threshold — boundary cases decide everything.

Counting the Damage: 75% Across the Board

TP = 3 · FP = 1 · FN = 1 · TN = 3 — now the three metrics

	Actual: Default	Actual: Safe
Predicted: Default	TP = 3 (A, C, F)	FP = 1 (D)
Predicted: Safe	FN = 1 (G)	TN = 3 (B, E, H)

The expensive mistake

- G is the FN — a defaulter approved; the bank eats the loss.
- Recall watches exactly this mistake.

$$\text{Acc} = \frac{3+3}{8} = 75\% \quad \text{Prec} = \frac{3}{4} = 75\% \quad \text{Rec} = \frac{3}{4} = 75\%$$

Why accuracy can lie (lecture link)

- Here all three metrics are 75% — a coincidence of balanced classes. With 1-in-1,000 fraud, a model that always says 'Safe' scores 99.9% accuracy and 0% recall. Accuracy flatters; recall protects the bank.

Scale It: From Eight Rows to Ten Thousand

Paste the metaprompt — Claude Code pits logistic regression against your hand rule

[INPUT] Prompt · Claude Code

```
Build a single Colab-ready Python script that:
1) simulates 10,000 borrowers with income, debt and a
   default label whose probability rises with 2*debt
   minus income plus noise (about 8% default rate);
   (offline classroom: load fml_w1_p3_credit_sim.csv)
2) splits 70/30 (stratified), fits LogisticRegression;
3) on the TEST set prints, for BOTH the model and the
   hand rule 'default iff 2*debt - income > 0':
   confusion matrix, accuracy, precision, recall, AUC;
4) then repeats the logistic fit on REAL public data:
   fetch_openml('credit-g') - free, no signup, no key -
   and prints the same metrics on its test split;
5) two-line verdict: which wins on recall, and why
   accuracy alone would have misled us.
```

[OUTPUT] What to expect

```
sim: logistic recall clearly
    above hand rule; AUC .75-.85
hand rule: decent accuracy,
    weaker recall
credit-g: AUC ~ .75-.80 on
    1,000 real borrowers

Learned weights beat guessed
weights - that is training.
```

Problem Set — Three Things Your Hands Now Know

You computed every number yourself; the code only repeats what you already understand

1

Fat tails: one crash day moves every moment

$S = -1.5$ and $K > 3$ came from a single -4% day — financial risk lives in the tails.

2

Overfitting: the train-test gap is the detector

Train MSE 0.00 vs test MSE 9.00 — a perfect training score is a warning, not a triumph.

3

Classification: accuracy flatters, recall protects

The missed defaulter (FN) is the expensive mistake — pick the metric that watches it.

Hand calculation first — then let Claude Code scale the same idea to real data.